每天只看这一页

仓库 Patick274/wheeltec_robot；分支 main；远程别名 origin。先完成首次配置；报错先停。

> **交接顺序**
>
> A（Windows）push → GitHub → B（Linux）pull；B 做完再 push，回 A 先 pull。

A / Windows	PowerShell，进入本地 PHM 文件夹；可继续使用 HTTPS。
B / Linux	Bash，教程目录 ~/projects/wheeltec_robot；采用 SSH，不用登录插件。

clone 首次克隆；pull 下拉并整合；commit 形成本地版本；push 上传提交。

01 开工：先看状态，拉取后再编辑

```
git branch --show-current
git status
```

确认当前 main，工作区干净且无进行中的合并后，再运行下面两条。

```
git pull --ff-only
git log -1 --oneline
```

核对另一台电脑的提交后，再编辑和保存。

02 收工：检查 → 暂存 → 提交 → 推送

```
git status
git diff
git add .
git diff --cached
git commit -m "说明本次修改"
git push
git log -1 --oneline
```

git add . 会暂存当前目录内所有未忽略修改；先确认文件范围。无新修改不必新 commit，已有提交仍可能待 push。按项目说明完成必要检查。

> **交接成功的标志**
>
> push 成功；网页选 main 看到提交短编号；另一台 pull 后核对同一提交。clean 只表示本地没有待提交修改，不代表已上传。

首次连接：A 上传，B 克隆

首次配置和每日操作分开；以下各段有不同前提，不整页一次执行。

A / Windows：已有本地提交 → 空远程

```
Set-Location -LiteralPath 'C:\Users\thunderobot\OneDrive\Desktop\项目\轮式升降机\PHM'
git status
git remote -v
git branch --show-current
```

仅当 remote -v 无输出时，添加一次：

```
git remote add origin https://github.com/Patick274/wheeltec_robot.git
```

已有正确 origin 就跳过 add；地址错误用 set-url，详见配套 MD。确认远程为空、本地 main 已有提交且无未完成合并后：

```
git push -u origin main
```

-u 成功后记住上游，以后通常 git push。已有提交无需为上传再造一次 commit；远程有历史先看第 4 页。

B / Linux：作者配置与首次 clone

先在 GitHub 网页确认 A 已上传；B 的 SSH 认证见第 3 页。检查 git --version；未安装时按发行版安装 Git。先替换邮箱再运行：

```
git config --global user.name "Patick274"
git config --global user.email "替换成你的提交邮箱"
```

```
mkdir -p ~/projects
cd ~/projects
git clone git@github.com:Patick274/wheeltec_robot.git
cd ~/projects/wheeltec_robot
git branch -vv
```

目标目录已存在就先核对，不覆盖。clone 已有 main 时通常自动配置 origin 和上游；不再 init。A/B 路径和认证方式可以不同。

> **三个变量，随时查**
>
> git remote -v 查别名和地址；git branch --show-current 查当前分支；git branch -vv 中 [origin/main] 表示上游已关联。

Linux 不登录插件：用 SSH

在 Linux 系统终端执行；GitHub 插件可以不登录，SSH 密钥负责仓库认证。

01 先检查，再按需生成本机密钥

```
ls -al ~/.ssh
```

已有 id_ed25519 和 .pub 可先评估复用；仅在没有时运行下一条。若提示覆盖先停，保存位置可用默认值，建议设置密钥口令。

```
ssh-keygen -t ed25519 -C "Patick274-linux"
```

02 加载密钥，复制公钥到 GitHub

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

复制输出的完整公钥。可在 A 已登录的浏览器操作：
Settings → SSH and GPG keys → New SSH key
类型 Authentication Key，标题可写 Linux电脑，粘贴公钥。只复制 .pub；私钥不上传、不复制到另一台电脑。

03 测试认证，再用 SSH clone / push

```
ssh -T git@github.com
```

首次连接核对 GitHub 官方主机指纹（点击），匹配后再确认。成功消息应含 Hi Patick274!；does not provide shell access 是正常提示。

> **出现 Permission denied (publickey)**
>
> 说明密钥认证尚未完成：核对公钥是否加到正确账号、agent 是否加载正确密钥。若成功消息显示别的账号，也先停止核对。

如果 B 已经用 HTTPS 克隆，进入那份仓库，仅将 origin 改为 SSH 地址：

```
git remote set-url origin git@github.com:Patick274/wheeltec_robot.git
```

设有口令的密钥可能询问本机口令；agent 会话结束后可能需要重新加载。这不要求登录 GitHub 插件。

遇到报错：查原因，不反复 push

你的实际问题速查。每段报错先停；特殊历史处理不混入每天的操作卡。

LF → CRLF warning	换行提示通常不代表 add 失败；用 status 和 diff --cached 核对，先不乱改全局配置。
remote -v 无输出	当前本地仓库无远程。核对目录后 add origin 一次；已有 origin 改地址用 set-url。
push 要求指定目标	远程和上游需先设置；首次成功 push -u origin main，之后通常直接 push。
Untracked / clean	新文件按需提交或忽略；clean / nothing to commit 只说明无待提交修改，仍可能待上传。
fetch first / 分叉	先看 status、fetch 和提交图；有共同历史时合并双方提交。--ff-only 分叉会停止。
unrelated histories	两边独立初始化，先备份并诊断；--allow-unrelated-histories 是特殊操作，不是每日参数。
CONFLICT (add/add)	同路径文件由双方分别新增。比较内容、清理标记、检查并暂存，再完成合并；不盲选一边。
新空仓库仍有旧冲突	仅明确放弃旧 merge 且已保留所需修改时用 merge --abort；确认恢复状态后再首次 push。
插件要求登录	Linux 用终端 Git + SSH；插件可不登录，仓库认证仍要完成。详见第 3 页。

历史与范围

你曾遇到两套独立历史和 5 个 add/add 冲突，后改用新空仓库。本手册只更新学习资料，不执行合并、提交或推送；不宣称 Linux 已配置成功。详细分流见配套 MD。

官方依据 / 点击打开

GitHub 克隆 / SSH 认证 / SSH 测试 / git pull / git merge / 换行配置

配套：code/03_另一台电脑拉取与上传/GitHub_AB两机同步简明流程.md
详细报错：code/04_常见问题/13_本次实际问题与处理.md